

06 — Application Architecture

Purpose

The purpose of this document is to define the application architecture of Mosaica.

This module establishes how user intentions are transformed into executable application workflows through the coordination between:

- API Layer,
- Application Layer,
- Domain Layer,
- Persistence Layer.

The Application Architecture defines the orchestration structure required to execute business operations while preserving Domain responsibilities.

Scope

This module defines:

- Application Layer responsibilities,
- Use Cases,
- Application Services,
- Workflow coordination,
- Transaction management,
- Application-level architectural decisions.

Architectural Position

API Layer

↓

Application Architecture

↓

Domain Layer

↓

Database Layer

Module Structure

06 Application Architecture

06.01 Application Layer Architecture

06.02 Use Case Design

06.03 Application Service Design

06.04 Transaction & Workflow Design

06.05 Application Architecture Review

06 — Application Architecture

Purpose

The purpose of this document is to define the architectural structure of the Mosaica Application Layer.

The Application Layer coordinates user intentions and domain operations while preserving the responsibilities of the Domain Model.

This layer acts as the bridge between external interfaces and business logic.

Application Architecture Principles

Principle 1 — Application Layer Orchestration

The Application Layer coordinates workflows.

It does not contain core business rules.

Principle 2 — Domain Ownership

Business decisions remain inside the Domain Layer.

The Application Layer delegates business behavior to Aggregates and Domain objects.

Principle 3 — Use Case Orientation

Application services are organized around user goals and business operations.

They are not organized around database tables.

Principle 4 — Transaction Coordination

The Application Layer defines transaction boundaries between operations while respecting Aggregate boundaries.

Principle 5 — Thin Application Layer

The Application Layer should coordinate, not become a second Domain Layer.

Application Layer Position

The Mosaica architecture follows this structure:

External Client

↓

API Layer

↓

Application Layer

↓

Domain Layer

↓

Persistence Layer

Layer Responsibilities

API Layer

Responsible for:

- receiving requests,
- authentication context,
- request formatting,
- response formatting.

Does not contain:

- business workflows,
 - domain decisions.
-

Application Layer

Responsible for:

- executing use cases,
 - coordinating Aggregates,
 - managing workflows,
 - controlling transaction execution.
-

Domain Layer

Responsible for:

- business rules,
 - invariants,
 - state transitions,
 - domain behavior.
-

Persistence Layer

Responsible for:

- data storage,
 - repositories,
 - database communication.
-

Application Service Concept

Application Services represent user intentions.

A service answers:

"What does the user want to accomplish?"

Examples:

Create Library Entry

Update Library Status

Record Consumption Event

Create Collection

Search Content

Application Service Responsibilities

An Application Service may:

- receive a request,
 - validate application-level requirements,
 - load required Aggregates,
 - execute domain operations,
 - save changes,
 - return results.
-

An Application Service shall not:

- calculate business rules,
 - modify Entity state directly,
 - bypass Aggregate Roots.
-

Example Workflow

Create Library Entry

User action:

"Add Content to my library."

Flow:

API

↓

CreateLibraryEntryService

↓

Load Content Aggregate

↓

Create Library Entry Aggregate

↓

Save Library Entry

↓

Return Response

Record Consumption Event

User action:

"I watched this movie."

Flow:

API

↓

RecordConsumptionService

↓

Load Library Entry Aggregate

↓

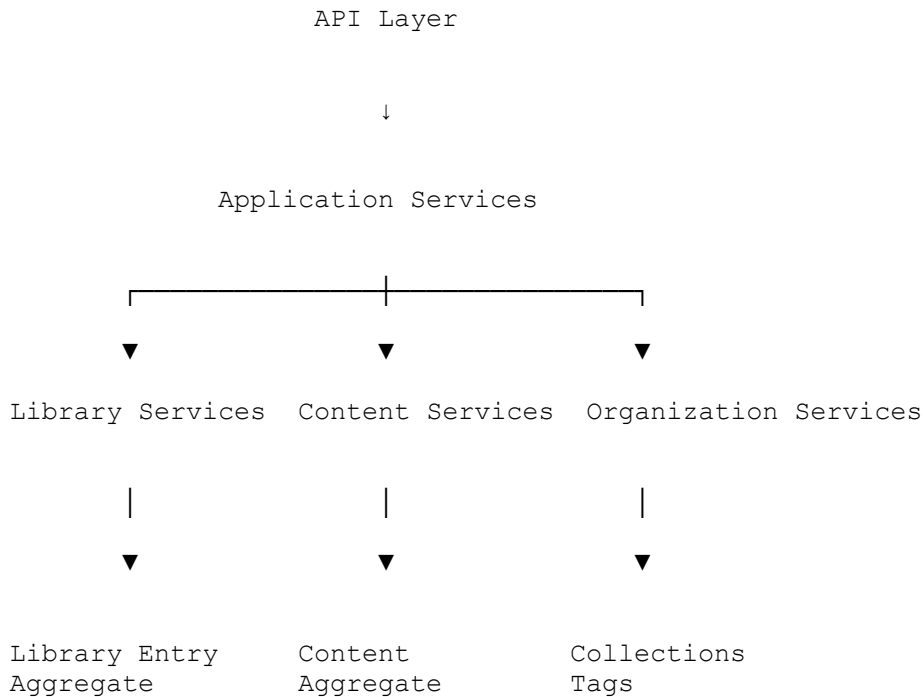
Add Consumption Event

↓

Save Aggregate

↓

Application Architecture Model



Application Service Categories

Mosaica Application Services are grouped according to domain responsibilities.

Library Application Services

Responsible for:

- Library Entry creation,
- personal state updates,
- consumption tracking.

Examples:

CreateLibraryEntryService

UpdateLibraryStatusService

UpdateRatingService

RecordConsumptionService

Content Application Services

Responsible for:

- Content management,
 - metadata operations.
-

Examples:

CreateContentService

UpdateContentService

SearchContentService

Organization Application Services

Responsible for:

- Collections,
 - Tags.
-

Examples:

CreateCollectionService

AddEntryToCollectionService

CreateTagService

Transaction Responsibility

Transactions shall be coordinated at the Application Layer.

However:

Aggregate consistency remains protected by the Domain Layer.

Example:

Application Layer

"Record Consumption"

↓

Library Entry Transaction

↓

Library Entry Aggregate

Architectural Decisions Confirmed

Decision 01

Application Layer coordinates use cases but does not contain business rules.

Decision 02

Use cases define Application Services.

Decision 03

Aggregates remain the only owners of business consistency.

Decision 04

Application Services communicate with Aggregates through their public interfaces.

Decision 05

Application Layer acts as the orchestration boundary between API and Domain.

06.02 — Use Case Design

Purpose

The purpose of this section is to define the official Use Cases of the Mosaica Application Layer.

Use Cases represent the business intentions that the system supports.

They describe what Users want to accomplish and how the Application Layer coordinates the required Domain operations.

Use Cases provide the connection between external requests and internal business behavior.

Use Case Design Principles

Principle 1 — User Goal Orientation

Each Use Case represents a meaningful User goal.

Use Cases shall not be created only because a database operation exists.

Principle 2 — Application Coordination

Use Cases define workflow coordination.

They do not contain core business rules.

Principle 3 — Domain Delegation

Business decisions remain inside Domain Aggregates and Domain Services.

Use Cases request operations; they do not replace Domain logic.

Principle 4 — Aggregate Respect

Each Use Case interacts with Aggregates through their defined boundaries.

Use Case Categories

Mosaica Use Cases are organized into four main categories.

Use Cases

- Identity Use Cases
 - Content Use Cases
 - Library Use Cases
 - Organization Use Cases
-

1. Identity Use Cases

UC-001 — Register User

Purpose

Creates a new User identity within the system.

Actor

User

Main Flow

User

↓

Registration Request

↓

Identity Validation

↓

User Creation

↓

Account Confirmation

Application Responsibility

- Coordinate registration workflow.
- Create User identity.

Domain Responsibility

- Protect User identity rules.

UC-002 — Authenticate User

Purpose

Allows an existing User to access the system.

Actor

User

Main Flow

Login Request

↓

Identity Verification

↓

Authentication Result

↓

User Session

2. Content Use Cases

UC-003 — Create Content

Purpose

Creates a new shared Content record.

Actor

Authorized User/System

Aggregate

Content Aggregate

Main Flow

Create Content Request

↓

Application Service

↓

Content Aggregate Creation

↓

Persistence

↓

Response

UC-004 — View Content

Purpose

Allows Users to access shared cultural metadata.

Actor

User

Main Flow

Search/View Request

↓

Content Retrieval

↓

Content Response

UC-005 — Update Content Metadata

Purpose

Updates information belonging to the Content Aggregate.

Aggregate

Content Aggregate

Important Rule

This Use Case cannot modify:

- User ratings,
 - Favorites,
 - Status,
 - Personal Notes,
 - Consumption History.
-

3. Library Use Cases

Library Use Cases represent the User's personal relationship with Content.

UC-006 — Add Content To Library

Purpose

Creates a personal Library Entry connecting a User with Content.

Actor

User

Aggregates

- Content Aggregate
 - Library Entry Aggregate
-

Main Flow

User selects Content

↓

Application Service

↓

Content existence check

↓

Library Entry creation

↓

Save Library Entry

Result

A new personal archive relationship is created.

UC-007 — View Personal Library

Purpose

Retrieves the User's personal archive.

Actor

User

Main Flow

Library Request

↓

User Context Validation

↓

Retrieve Library Entries

↓

Return Archive

UC-008 — Update Library Status

Purpose

Changes the workflow state of a Library Entry.

Example

Planned

↓

Watching

↓

Completed

Aggregate

Library Entry Aggregate

Domain Responsibility

The Domain decides whether the transition is valid.

UC-009 — Update Personal Evaluation

Purpose

Updates User evaluation of Content.

Includes:

- Rating
 - Appreciation Level
-

Aggregate

Library Entry Aggregate

UC-010 — Update Personal Archive State

Purpose

Manages personal organization state.

Includes:

- Favorite
 - Archive Location
-

Aggregate

Library Entry Aggregate

UC-011 — Record Consumption Event

Purpose

Records a User's experience with Content.

Aggregate

Library Entry Aggregate

Main Flow

User records experience

↓

Application Service

↓

Load Library Entry

↓

Add Consumption Event

↓

Save Aggregate

UC-012 — Manage Personal Notes

Purpose

Creates or updates personal reflections about Content.

Aggregate

Library Entry Aggregate

4. Organization Use Cases

UC-013 — Create Collection

Purpose

Creates a User-defined organization group.

Actor

User

Result

A new Collection is created.

UC-014 — Add Library Entry To Collection

Purpose

Associates a Library Entry with a Collection.

Important Rule

Collection organizes Library Entries.

It does not own them.

UC-015 — Create Tag

Purpose

Creates a User-defined classification label.

UC-016 — Tag Library Entry

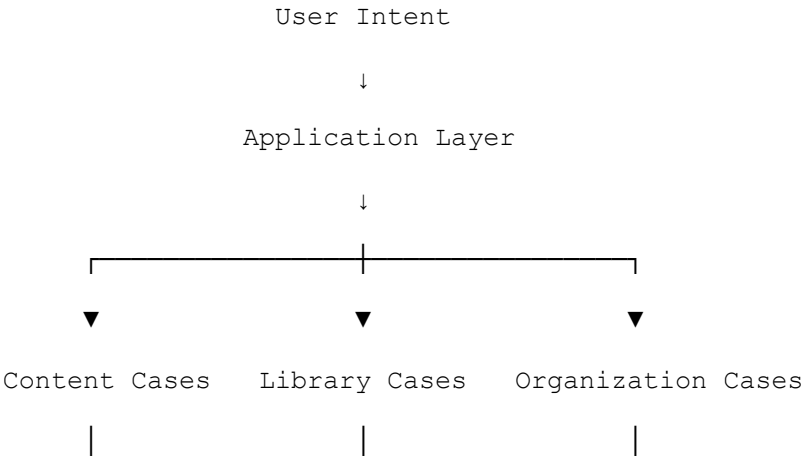
Purpose

Associates a Tag with a Library Entry.

Use Case Overview

ID	Use Case	Aggregate
UC-001	Register User	User
UC-002	Authenticate User	User
UC-003	Create Content	Content
UC-004	View Content	Content
UC-005	Update Content Metadata	Content
UC-006	Add Content To Library	Library Entry
UC-007	View Personal Library	Library Entry
UC-008	Update Library Status	Library Entry
UC-009	Update Personal Evaluation	Library Entry
UC-010	Update Personal Archive State	Library Entry
UC-011	Record Consumption Event	Library Entry
UC-012	Manage Personal Notes	Library Entry
UC-013	Create Collection	Collection
UC-014	Add Entry To Collection	Collection
UC-015	Create Tag	Tag
UC-016	Tag Library Entry	Tag

Application Layer Use Case Model



▼
Content Aggregate

▼
Library Entry
Aggregate

▼
Collections
Tags

Architectural Decisions Confirmed

Decision 01

Use Cases represent User goals, not database operations.

Decision 02

Application Layer coordinates Use Cases.

Decision 03

Business rules remain inside the Domain Layer.

Decision 04

Library Entry Aggregate manages personal experience workflows.

Decision 05

Content Aggregate manages shared cultural metadata.

06.03 — Application Service Design

Purpose

The purpose of this section is to define the official Application Service architecture of Mosaica.

Application Services are responsible for coordinating Use Cases between external interfaces and Domain Aggregates.

They provide the execution layer where user intentions are transformed into domain operations.

Application Services do not contain business rules.

Application Service Principles

Principle 1 — Use Case Alignment

Each Application Service shall represent one or more clearly defined Use Cases.

Services shall exist because a business operation exists.

Principle 2 — Domain Delegation

Application Services coordinate operations but do not implement business decisions.

Business behavior remains inside Domain Aggregates.

Principle 3 — Aggregate Respect

Application Services interact with Aggregates through their public interfaces.

They shall not directly modify internal Entity state.

Principle 4 — Single Responsibility

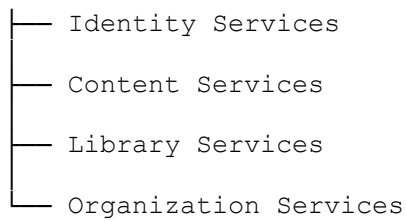
Each Application Service shall have a focused responsibility.

Large services containing unrelated workflows shall be avoided.

Application Service Architecture

The Mosaica Application Layer is organized into service groups.

Application Services



1. Identity Application Services

RegisterUserService

Related Use Case

UC-001 — Register User

Responsibility

Coordinates User registration workflow.

Main Flow

Registration Request

↓

Validate Application Requirements

↓

Create User

↓

Persist User

↓

Return Result

Does Not

- manage authentication rules,
- manage personal archive data.

AuthenticateUserService

Related Use Case

UC-002 — Authenticate User

Responsibility

Coordinates User authentication workflow.

Main Flow

Login Request

↓

Verify Identity

↓

Create Authentication Context

↓

Return Authentication Result

2. Content Application Services

CreateContentService

Related Use Case

UC-003 — Create Content

Responsibility

Creates a new Content Aggregate.

Main Flow

Create Request

↓

Validate Input

↓

Create Content Aggregate

↓

Save Content

↓

Return Content Result

ViewContentService

Related Use Case

UC-004 — View Content

Responsibility

Retrieves shared Content information.

UpdateContentService

Related Use Case

UC-005 — Update Content Metadata

Responsibility

Coordinates Content metadata updates.

Aggregate

Content Aggregate

3. Library Application Services

Library Services represent the core User experience workflows.

CreateLibraryEntryService

Related Use Case

UC-006 — Add Content To Library

Responsibility

Creates a User's personal relationship with Content.

Main Flow

Request

↓

Identify User

↓

Validate Content Exists

↓

Create Library Entry Aggregate

↓

Save Library Entry

↓

Return Result

GetLibraryService

Related Use Case

UC-007 — View Personal Library

Responsibility

Retrieves User-owned Library Entries.

Security Requirement

Only authenticated User data may be returned.

UpdateLibraryStatusService

Related Use Case

UC-008 — Update Library Status

Responsibility

Requests a status change on a Library Entry.

Domain Responsibility

The Library Entry Aggregate determines whether the transition is valid.

UpdateEvaluationService

Related Use Case

UC-009 — Update Personal Evaluation

Responsibility

Coordinates updates to:

- Rating,
 - Appreciation Level.
-

UpdateArchiveStateService

Related Use Case

UC-010 — Update Personal Archive State

Responsibility

Coordinates:

- Favorite changes,
 - Archive Location changes.
-

RecordConsumptionService

Related Use Case

UC-011 — Record Consumption Event

Responsibility

Adds a new Consumption Event to a Library Entry.

Main Flow

Consumption Request

↓

Load Library Entry Aggregate

↓

Add Consumption Event

↓

Save Aggregate

↓

Return Result

Important Rule

Consumption Event creation is not independent.

It always occurs through Library Entry.

UpdatePersonalNoteService

Related Use Case

UC-012 — Manage Personal Notes

Responsibility

Coordinates personal note updates.

4. Organization Application Services

CreateCollectionService

Related Use Case

UC-013 — Create Collection

Responsibility

Creates User-owned organizational groups.

AddEntryToCollectionService

Related Use Case

UC-014 — Add Library Entry To Collection

Responsibility

Creates organization relationships.

Important Rule

The service does not transfer ownership.

Collection only organizes Library Entries.

CreateTagService

Related Use Case

UC-015 — Create Tag

Responsibility

Creates User-defined classification labels.

TagLibraryEntryService

Related Use Case

UC-016 — Tag Library Entry

Responsibility

Creates Tag relationships.

Application Service Map

Use Case	Application Service
Register User	RegisterUserService
Authenticate User	AuthenticateUserService
Create Content	CreateContentService
View Content	ViewContentService
Update Content	UpdateContentService
Add Content To Library	CreateLibraryEntryService
View Personal Library	GetLibraryService
Update Status	UpdateLibraryStatusService
Update Evaluation	UpdateEvaluationService
Update Archive State	UpdateArchiveStateService
Record Consumption	RecordConsumptionService
Manage Notes	UpdatePersonalNoteService
Create Collection	CreateCollectionService
Add Entry To Collection	AddEntryToCollectionService
Create Tag	CreateTagService
Tag Entry	TagLibraryEntryService

Application Service Flow Model

API Request

↓

Application Service

↓

Repository

↓

Aggregate

↓

Domain Operation

↓

Persistence

Architectural Decisions Confirmed

Decision 01

Application Services represent Use Case execution.

Decision 02

Application Services coordinate but do not contain business rules.

Decision 03

Aggregates remain responsible for business consistency.

Decision 04

Each major user intention has a dedicated application workflow.

Decision 05

Consumption Events are always managed through Library Entry workflows.

06.04 — Transaction & Workflow Design

Purpose

The purpose of this section is to define the transaction and workflow behavior of the Mosaica Application Layer.

This section establishes:

- transaction boundaries,
- workflow execution order,
- Aggregate interaction rules,
- consistency management principles.

The objective is to ensure that application workflows preserve Domain integrity while maintaining scalable architecture.

Transaction Design Principles

Principle 1 — Aggregate-Based Transactions

Transactions shall primarily follow Aggregate boundaries.

A single business operation should modify one Aggregate whenever possible.

Principle 2 — Application Layer Coordination

The Application Layer coordinates transactions.

It does not replace Aggregate consistency rules.

Principle 3 — Avoid Distributed Transactions

Operations requiring multiple Aggregates shall avoid large transactional boundaries.

Cross-Aggregate coordination shall be handled through application workflows.

Principle 4 — Consistency Over Convenience

The architecture prioritizes correct business state over simplified database operations.

Transaction Boundary Model

The primary transaction boundaries are:

Content Aggregate

Transaction Boundary

Library Entry Aggregate

Transaction Boundary

Each Aggregate protects its own consistency.

Library Entry Transaction Model

The Library Entry Aggregate is responsible for personal archive operations.

Examples:

- Status changes
 - Rating updates
 - Favorite changes
 - Personal Notes
 - Consumption Events
-

Example: Update Library Status

Workflow:

User Request

↓

UpdateLibraryStatusService

↓

Load Library Entry Aggregate

↓

Request Status Change

↓

Validate Domain Rule

↓

Save Library Entry

↓

Return Result

Transaction Boundary

The transaction begins when the Library Entry Aggregate is loaded.

The transaction ends after the Aggregate state is successfully persisted.

Example: Record Consumption Event

Workflow:

User Request

↓

RecordConsumptionService

↓

Load Library Entry Aggregate

↓

Create Consumption Event

↓

Add Event To Aggregate

↓

Save Library Entry

↓

Return Result

Transaction Boundary

The Consumption Event creation belongs to the Library Entry transaction.

The event does not create its own independent transaction.

Content Transaction Model

The Content Aggregate manages shared cultural metadata.

Examples:

- Create Content
 - Update Content Metadata
-

Example: Create Content

Workflow:

Create Request

↓

CreateContentService

↓

Create Content Aggregate

↓

Validate Domain State

↓

Save Content

↓

Return Result

Transaction Boundary

The transaction belongs entirely to the Content Aggregate.

Cross-Aggregate Workflow Design

Some operations involve more than one Aggregate.

Example:

Add Content To Library

This operation involves:

- Content Aggregate
- Library Entry Aggregate

Workflow

User Request

↓

CreateLibraryEntryService

↓

Verify Content Exists

↓

Create Library Entry Aggregate

↓

Save Library Entry

↓

Return Result

Important Decision

Content is not modified.

Only existence is verified.

Therefore:

The transaction belongs only to Library Entry.

Why?

Incorrect approach:

Content Transaction

+

Library Entry Transaction

+

Single Large Transaction

This creates unnecessary coupling.

Correct approach:

Read Content

↓

Create Library Entry

↓

Persist Library Entry

Organization Workflow Design

Collections and Tags operate as organizational features.

Add Entry To Collection

Workflow:

Request

↓

AddEntryToCollectionService

↓

Verify Ownership

↓

Create Relationship

↓

Save Relationship

Transaction Boundary

The relationship operation is handled independently.

Library Entry ownership does not transfer.

Tag Library Entry

Workflow:

Request

↓

TagLibraryEntryService

↓

Verify Ownership

↓

Create Tag Relationship

↓

Save Relationship

Failure Handling During Workflows

If an operation fails:

- incomplete Aggregate changes are discarded,
 - invalid state is not persisted,
 - the client receives a standardized API error.
-

Workflow State Model

Application workflows follow this pattern:

Receive Request

↓

Authenticate User

↓

Authorize Action

↓

Load Required Data

↓

Execute Domain Operation

↓

Persist Changes

↓

Return Response

Transaction Responsibility Matrix

Operation	Main Aggregate	Transaction Boundary
Create Content	Content	Content Aggregate
Update Content	Content	Content Aggregate
Add Library Entry	Library Entry	Library Entry Aggregate
Update Status	Library Entry	Library Entry Aggregate
Update Rating	Library Entry	Library Entry Aggregate
Record Consumption	Library Entry	Library Entry Aggregate
Create Collection	Collection	Collection Operation
Add Entry To Collection Relationship		Relationship Operation

Architectural Decisions Confirmed

Decision 01

Transactions follow Aggregate boundaries.

Decision 02

Library Entry owns personal workflow consistency.

Decision 03

Consumption Events are persisted within Library Entry workflows.

Decision 04

Cross-Aggregate operations avoid unnecessary distributed transactions.

Decision 05

Application Services coordinate workflows without owning business rules.

06.05 — Application Architecture Review

Review Objective

The purpose of this Architecture Review is to validate that the Mosaica Application Architecture correctly defines the coordination layer between API, Domain and Persistence layers.

This review confirms that the Application Layer preserves:

- Domain responsibility,
 - Aggregate boundaries,
 - workflow consistency,
 - transaction integrity,
 - architectural simplicity.
-

Scope Reviewed

The following components have been reviewed:

- Application Layer Architecture
- Use Case Design
- Application Service Design
- Transaction & Workflow Design

Validation Results

Application Layer Responsibility

Status: Approved

The Application Layer correctly acts as an orchestration layer between external communication and Domain operations.

It does not contain:

- business rules,
- persistence logic,
- domain state management.

Use Case Alignment

Status: Approved

Use Cases correctly represent User goals and business intentions.

Use Cases are not derived from database operations.

The architecture preserves the distinction:

User Goal

↓

Use Case

↓

Application Service

↓

Domain Operation

Application Service Responsibility

Status: Approved

Application Services correctly coordinate workflows.

They are responsible for:

- loading required data,
- invoking Domain operations,
- managing application flow,
- coordinating persistence.

They do not replace Domain logic.

Domain Protection

Status: Approved

Business decisions remain inside the Domain Layer.

Application Services cannot bypass:

- Aggregate rules,
 - Entity behavior,
 - Value Object validation.
-

Aggregate Boundary Protection

Status: Approved

Application workflows respect approved Aggregate boundaries.

Confirmed:

Content Aggregate

Responsible for:

- shared cultural metadata,
- Content-related operations.

Library Entry Aggregate

Responsible for:

- personal archive state,
 - evaluation,
 - workflow,
 - consumption history.
-

Transaction Design

Status: Approved

Transaction boundaries correctly follow Aggregate boundaries.

The architecture avoids unnecessary distributed transactions.

Cross-Aggregate Operations

Status: Approved

Operations involving multiple Aggregates are coordinated without creating excessive coupling.

Example:

Adding Content to Library:

Content

↓

Existence Check

↓

Library Entry Creation

Content is referenced, not modified.

Workflow Consistency

Status: Approved

Application workflows follow a consistent execution model:

Request

↓

Authentication

↓

Authorization

↓

Application Service

↓

Domain Operation

↓

Persistence

↓

Response

Architectural Decisions Confirmed

Decision 01

The Application Layer coordinates business workflows but does not own business rules.

Decision 02

Use Cases define system capabilities from a User perspective.

Decision 03

Application Services implement Use Case orchestration.

Decision 04

Aggregates remain responsible for maintaining business consistency.

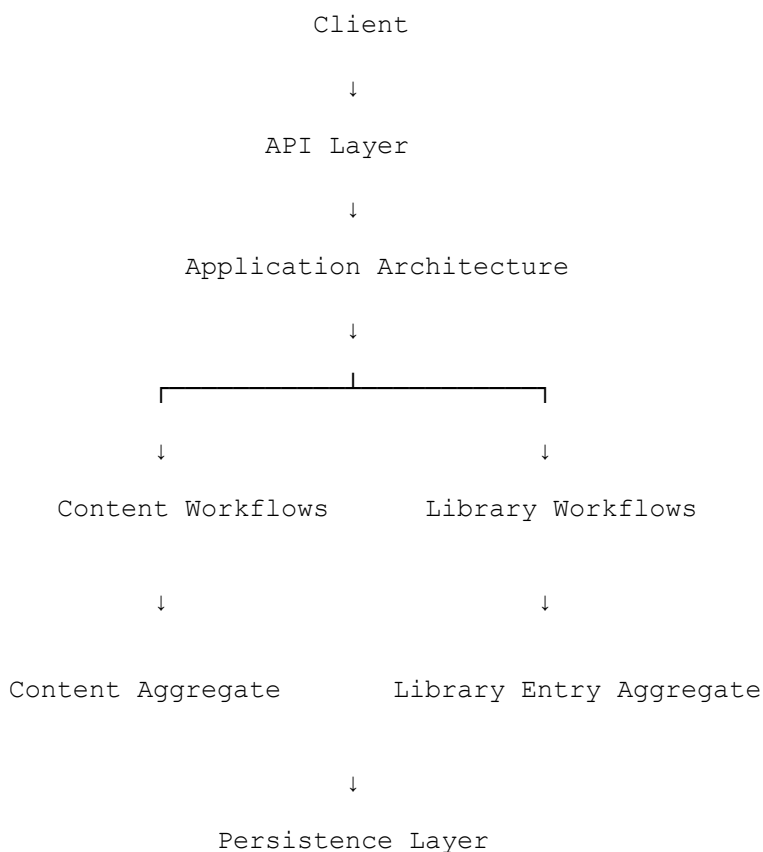
Decision 05

Transactions primarily follow Aggregate boundaries.

Decision 06

Cross-Aggregate workflows avoid unnecessary coupling.

Application Architecture Summary



Review Conclusion

The Mosaica Application Architecture is considered architecturally complete.

The current design establishes a clear separation between:

- external communication,
- workflow coordination,

- business logic,
- data persistence.

The Application Layer provides a scalable foundation for implementing future features while preserving Domain integrity.

No blocking architectural issues have been identified.